KingdomForge: Benchmark Architecture & Live-Run Hardening Specification

Benchmark v2 System Design Document | Autonomous RL Environment for LLM Capability Assessment

1. Environment & Capability-Centric Task Design

KingdomForge is a deterministic, discrete-event medieval realm management simulation designed to benchmark LLM planning, resource stewardship, and error adaptation under stringent live-run constraints. The suite comprises 13 distinct capability-oriented tasks across two frozen tiers:

Task & Capability Dimension	Core Mechanics & Decision Primitives	Verification & Termination Criteria
Tasks 1–6 (Frozen v1 Regression Suite) Foundational resource economy, trade, population scaling, defensive raids, balanced kingdoms.	Market trading, worker assignment (wood/stone/food), food consumption per turn, periodic bandit raids requiring wall/tower defense.	Deterministic state assertions. Reference baseline ≥ 0.95, No-op baseline ≤ 0.05. Max 25–40 turns.
Task 7: The Mason's Citadel Dependency & Action Ordering	Strict prerequisite hierarchy: Workshop → Masonry → Stone Wall → Castle. Wall stops raids; premature stone sales cause deadlock.	Actual castle construction, active stone wall, zero raid collapse. Verified against environment state.
Task 8: Scarce-Resource Budgeting Austerity & Opportunity Cost	Zero starting gold, 25 initial food, zero natural gold nodes. Requires early lumber mill to generate exportable timber.	4+ settlement buildings, food storage ≥ 20, positive treasury, zero starvation events.
Task 9: The Long Horizon Multi-Turn Temporal Planning	15-turn minimum survival horizon. Population grows consuming 1 food/person/turn. Worker allocation must prevent supply cliff.	Turn ≥ 15 reached, 4+ buildings, stone ≥ 30, gold ≥ 75, strictly zero starvation occurrences.
Task 10: Capacity Expansion Population & Housing Trade-offs	Housing caps total population. Recruiting workers increases consumption but enables multi-sector assignments.	Population ≥ 9, total capacity ≥ 12, 4+ buildings, stable food reserves (≥ 20).
Task 11: Adaptive Defense Recovery Fault Diagnosis & Replanning	Initial state with 0 stone, destroyed walls, pending bandit raid. Direct wall construction fails; model must mine stone first.	Quarry built, workers assigned, stone mined, defense raised ≥ 10, bandit raid repelled.
Task 12: Crown of the Realm Integrated Constrained Planning	Unified challenge requiring simultaneous optimization of agriculture, defense, masonry prerequisites, and prestige castle.	Castle built, defense ≥ 15, population ≥ 8, food ≥ 30, gold ≥ 25.
Task 13: Royal Economic Optimization Optimality Awareness & Epistemic Limits	Trade-off between gold surplus and defensive security. Feasibility threshold required before wealth points award.	Independent scoring rubric rewarding demonstrated feasibility and verifiable stopping claims over unfounded assertions.

2. State Representation, Action Contracts, & Explainable Rewards

State Invariants: Kingdom state comprises resources (gold, food, wood, stone), population (total, idle, capacity), buildings, worker assignments, defense rating, turn index, and event log. The environment enforces strict physical invariants on every mutation: non-negative resources, population conservation (total == idle + sum(assigned)), building worker limits, and non-negative defense rating.

Explainable Reward Decomposition: Reward is computed via structured programmatic verifiers, decomposing total return into four distinct, auditable components: (1) *Objective Completion* (weighted sum of explicit milestones), (2) *Constraint Validity* (adherence to resource and survival bounds), (3) *Progress Metric* (intermediate state milestones), and (4) *Integrity & Invariant Penalty* (forces score to 0.0 upon detected invariant corruption). Every reward breakdown records expected versus actual values.

Rigorous Failure Taxonomy: Failure modes are strictly partitioned: task_failure (agent did not meet goal), invalid_action (tool contract violation), tool_execution_failure (timeout/internal error), harness_failure (API/network crash), verifier_failure (scoring exception), and integrity_violation (state corruption).

3. Programmatic Verifier & Discrepancy Detection Engine

The verifier operates in complete physical isolation from agent tool execution. It directly reads SQLite/memory environment state and completely ignores model-generated self-claims. Success requires objective state proof. Discrepancy detection runs continuously across every step:

- **Error-with-Mutation Detection:** If a tool returns an error message but state hash mutates, a high-severity discrepancy is logged.
- **Invariant Violation Trap:** Any invariant breach instantly triggers integrity_violation, nullifying all accumulated provisional rewards to 0.0.
- **Provisional vs. Final Parity:** Per-step provisional rewards are recomputed independently; terminal bonuses cannot be collected without final verification.
- **Actionable Error Feedback:** Rejected actions emit structured diagnostics: INVALID_ACTION: <tool> | Reason: <explanation> | Constraint: <rule> | State change: none | Recovery: <safe next step> without leaking private solutions.

4. Live-Run Hard Limits & Concurrency Protections

To guarantee 100% adherence to the evaluator's automated test protocol, limits are enforced directly within the Python harness using monotonic clocks (time.monotonic()):

Evaluation Constraint	Hard Limit	Harness Enforcement Mechanism
Clean Docker Build	3 minutes (180s)	Cached build layers, minimal python-slim image, deterministic dependencies.
Combined Baseline Checks	2 minutes (120s)	Combined monotonic timeout; stops remaining execution if threshold breached.
Model Episode Wall-Clock	10 minutes (600s)	Monotonic elapsed time check on every step; terminates with time_cap.
Model Replies per Episode	40 replies	Strict step loop termination; text-only or invalid replies consume 1 step.
Tool Calls per Reply	10 calls max	Calls index ≥ 10 rejected with explicit diagnostic feedback.
Single Tool Call Duration	60 seconds	ThreadPoolExecutor isolated snapshot execution; non-blocking shutdown.
Final Verifier Scoring	2 minutes (120s)	Isolated pure programmatic calculation with execution timer guard.
Entire Live Slot Budget	16 minutes (960s)	Global slot monotonic hard stop enforced across baselines, model run, & scoring.

5. Threat Model, Defences, & Empirical Validation

Attack Surface & Defences: Model output is treated as untrusted payload. Arguments are strictly schema-validated via Pydantic/JSON parsers. Rogue mutations from timed-out threads are isolated by executing tool calls on sandboxed deep copies and committing state only upon timely return. Secret credentials (OPENAI_API_KEY, BASE_URL tokens) are sanitized via regex redaction before logging to JSONL. Missing APPEND_FILE immediately triggers safe exit code 3 (EXIT_CONFIG_ERROR).

Empirical Results & Baseline Gate Parity: Across all 13 tasks (26 baseline evaluations), KingdomForge demonstrates 100% gate compliance: every Reference Agent scores 1.0000 (≥ 0.95), every No-op Agent scores 0.0000 (≤ 0.05). Full unit test discovery verifies 112/112 passing automated test cases with zero regressions across Benchmark v1 tasks.

Total Tasks: 13	**Unit Tests:** 112 Passed / 0 Failed	**Baseline Gate:** 26/26 Passed (100%)	**Combined Baselines:** 0.08s (Limit: 120s)